B. M. S. COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to VTU, Belagavi

DEPARTMENT OF MACHINE LEARNING

Program: B.E. In Artificial Intelligence and Machine Learning

Academic Year: 2026 - 27, Session: July 2026 - Dec 2026

Natural Language Processing for LLM (24AM7PCLLM)

Alternative Assessment Tool (AAT) Report

Topic: Sentiment Analysis of Movie Reviews: Lexicon, Classical ML and Transformer Embeddings

Submitted by

Student Name: Team Member 1: Team Member 2:	
USN: Team Member 1: Team Member 2:	
Semester & Section:	

Valuation Report

(to be filled by faculty)

Score:	
Comments:	
Faculty In-charge:	Dr Monika Puttaramaiah
Faculty Signature with date:	

Table of Contents

1. Introduction

1.1 Background of Natural Language Processing
1.2 Importance of the Selected NLP Topic
1.3 Objective of the Study
1.4 Scope of the Report

2. Selected NLP Topic Overview

2.1 Description of the NLP Task (e.g., Text Classification, NER, MT, Chatbot)
2.2 Real-World Use Cases
2.3 Relevance to the Syllabus and Industry
2.4 Key Challenges in This NLP Task

3. Real-Time Application / API Identified

3.1 Application or API Name
3.2 Purpose and Functionality
3.3 Architecture or Platform on Which It Is Built
3.4 Input/Output Behavior
3.5 Limitations and Practical Considerations

4. NLP Model / Architecture Study

4.1 Theoretical Foundations of the Model
4.2 Algorithm(s) Used (e.g., Naive Bayes, LSTM, BERT, Transformers)
4.3 Architecture Diagram / Layer-Wise Design
4.4 Data Flow in the Model
4.5 Mathematical Formulation (if applicable)
4.6 Why This Model Suits the Selected NLP Task

5. Dataset and Preprocessing

5.1 Dataset Description (Source, Size, Structure)
5.2 Data Cleaning Steps
5.3 Tokenization, Lemmatization, Stopwords Removal
5.4 Train/Test Split Strategy
5.5 Feature Engineering or Embedding Approach (TF-IDF, Word2Vec, BERT Embeddings)

6. Implementation Methodology

6.1 Tools, Libraries, and Environment Used
6.2 Model Implementation Steps
6.3 API Integration (if used)
6.4 Pseudocode / Workflow Diagram
6.5 Hyperparameters and Training Configuration

7. Results and Evaluation

7.1 Performance Metrics (Accuracy, F1-Score, BLEU, Precision/Recall, etc.)
7.2 Visualization of Results (Confusion Matrix, Graphs)
7.3 Qualitative and Quantitative Analysis
7.4 Error Analysis
7.5 Comparison Between Approaches (If multiple methods used)

8. Discussion

8.1 Interpretation of Results
8.2 Strengths and Weaknesses of the Model
8.3 Challenges Encountered During Implementation
8.4 Impact of Dataset Quality and Preprocessing
8.5 How API or Application Enhances the NLP Task

9. Conclusion

9.1 Summary of the Work Done
9.2 Key Learnings and Insights
9.3 Real-World Applicability
9.4 Future Improvements or Extensions

10. References

1. Introduction

1.1 Background of Natural Language Processing

Natural Language Processing (NLP) enables computers to analyse and generate human language. The field moved from hand-written rules and lexicons, through statistical machine learning on bag-of-words features (Naive Bayes, Logistic Regression, Support Vector Machines), to neural representations: static word embeddings such as word2vec and GloVe, and contextual embeddings from Transformer models such as BERT and RoBERTa, which form the basis of today's Large Language Models (LLMs). Each generation trades interpretability and cost for accuracy, and choosing the right one for a task requires measuring them side by side.

1.2 Importance of the Selected NLP Topic

Sentiment analysis, the task of deciding whether a text expresses a positive or negative opinion, is one of the most widely deployed NLP applications. Companies track customer reviews and social media to measure satisfaction, film studios and streaming platforms analyse audience reaction, and investors monitor news sentiment. Short opinion sentences are linguistically hard: meaning depends on negation ('not good'), contrast ('stunning visuals, but a dull story'), intensifiers and sarcasm. This makes sentiment analysis an ideal task for comparing lexicon-based, statistical and Transformer-based approaches covered in the 24AM7PCLLM syllabus, and for showing where each succeeds or fails. Building reliable opinion-mining tools for industry supports SDG 9.5 (upgrading technological capabilities and encouraging innovation).

1.3 Objective of the Study

- Study the sentiment-analysis task and identify a real-time application and APIs that perform it.
- Apply Unit 1 preprocessing (cleaning, tokenisation, stopword removal, negation handling) and measure its effect with cross-validation.
- Implement and compare seven approaches: a rule-based lexicon (VADER), three classical classifiers on bag-of-words/TF-IDF features, static word embeddings, RoBERTa contextual embeddings and an ensemble.
- Evaluate with accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, training time and inference speed, and test statistical significance (McNemar's test).
- Analyse linguistic patterns (negation, contrast, intensifiers, sentence length, sarcasm) and data efficiency (learning curves).
- Deliver a working command-line sentiment analyser that explains its decisions.

1.4 Scope of the Report

The study uses the public Sentence Polarity dataset v1.0 of Pang and Lee (2005): 10,662 movie-review sentences labelled positive or negative. All models run on a CPU. Transformer embeddings are used frozen (feature extraction) rather than fine-tuned because no GPU was available; fine-tuning, multi-class (neutral) sentiment, aspect-based sentiment and Indian-language reviews are outside the scope and listed as future work.

2. Selected NLP Topic Overview

2.1 Description of the NLP Task (e.g., Text Classification, NER, MT, Chatbot)

Sentiment analysis is a **binary text-classification** task: given a sentence x, predict a label y ∈ {negative, positive}. A classifier learns a function f(x) = P(y = positive | x) from labelled examples, and predicts positive when f(x) ≥ 0.5. Lexicon-based systems instead sum the prior polarity of known words and adjust it with rules for negation, intensifiers and punctuation.

2.2 Real-World Use Cases

- **Product and service reviews:** e-commerce and food-delivery platforms flag negative reviews for follow-up and summarise satisfaction trends.
- **Brand and social-media monitoring:** companies track opinion about launches and campaigns in real time.
- **Film and entertainment:** review aggregators and studios measure critic and audience reaction (the domain of this project's dataset).
- **Customer support:** tickets and chats with strong negative sentiment are prioritised.
- **Finance:** sentiment of news headlines and reports is used as a market signal.

2.3 Relevance to the Syllabus and Industry

Table 2.1: Mapping of the project to the 24AM7PCLLM syllabus

Syllabus unit	Concept used in this project
Unit 1	Regular expressions, normalisation, tokenisation (incl. RoBERTa's byte-level BPE), stopwords, negation handling; ambiguity (why NLP is hard)
Unit 2	Bag-of-words, TF-IDF weighting, n-grams, static word embeddings, contextual embeddings
Unit 3	N-gram features, pre-trained language model RoBERTa, Transformer self-attention
Unit 4	Case study of an NLP/LLM-powered application (opinion mining)
Unit 5	Deployment trade-offs (speed vs accuracy), responsible AI: bias and misclassification risk

In industry, sentiment analysis is offered as managed APIs (Google Cloud Natural Language, Amazon Comprehend, Azure AI Language) and is a standard benchmark task for language models (the Stanford Sentiment Treebank in GLUE is derived from the same movie-review source used here).

2.4 Key Challenges in This NLP Task

- **Negation:** 'not good' and 'not bad at all' reverse the polarity of the word that follows.
- **Contrast:** 'the visuals are stunning, but the story never comes alive' — the clause after 'but' usually decides the overall sentiment.
- **Sarcasm and irony:** 'oh great, another two hours of my life I will never get back' uses positive words to express a negative opinion.

- **Domain-specific vocabulary:** words like 'unpredictable' are positive for a plot but negative for a car; general-purpose lexicons miss film-specific usage.
- **Short context:** sentences average only about 21 words, so there is little evidence per example.

3. Real-Time Application / API Identified

3.1 Application or API Name

Movie-Review Sentiment Analyser: a command-line application (predict.py) that classifies a review sentence as positive or negative, reports the probability, and lists the words that drove the decision. It uses the following APIs and libraries: **NLTK VADER** (an off-the-shelf rule-based sentiment API, used as the baseline), **scikit-learn** (TF-IDF, Naive Bayes, Logistic Regression, Linear SVM), **spaCy 3.8** (en_core_web_md word vectors) and the **RoBERTa-base** Transformer through spaCy's en_core_web_trf pipeline (PyTorch back-end). Cloud APIs such as Google Cloud Natural Language (analyzeSentiment) and Amazon Comprehend (DetectSentiment) provide the same function as a paid web service.

3.2 Purpose and Functionality

- Classify a review sentence as positive or negative with a confidence score.
- Explain the decision by listing the words with the largest positive or negative contribution.
- Show the VADER score alongside for comparison with a lexicon baseline.
- Offer a fast mode (TF-IDF only, no deep-learning dependency) and an accurate mode (ensemble with RoBERTa).

3.3 Architecture or Platform on Which It Is Built

The application is written in Python 3 and runs on CPU. At start-up it trains the TF-IDF + Logistic Regression model on the training split (about 2 seconds) and, in accurate mode, a Logistic Regression on cached RoBERTa sentence embeddings. Each input sentence is tokenised, vectorised by both feature extractors, scored by both classifiers and the two probabilities are averaged. The same functions can be exposed behind a REST endpoint for real-time use. The full pipeline is shown in Figure 4.1.

3.4 Input/Output Behavior

Table 3.1: Input-processing-output stages of the application

Stage	Description	Example
Input	One review sentence in English	"a masterpiece of quiet, heartbreaking performances."
Processing	Clean → tokenise → TF-IDF features + RoBERTa embedding → two Logistic Regression classifiers → average probability	TF-IDF LR 0.99, RoBERTa LR 1.00
Output	Label, P(positive), key words, VADER baseline score	POSITIVE (0.99); key words: masterpiece (+), performances (+)

3.5 Limitations and Practical Considerations

- Trained only on movie-review sentences from professional critics; casual or other-domain text (for example product reviews written informally) can be misclassified.
- Binary output: neutral or mixed sentences are forced into one class.
- RoBERTa embedding takes about 50 ms per sentence on CPU, roughly 2,000 times slower than TF-IDF.
- Sarcasm is not handled reliably by any of the models tested.

4. NLP Model / Architecture Study

4.1 Theoretical Foundations of the Model

Three families of sentiment models are studied. **Lexicon-based** methods (VADER, Hutto and Gilbert, 2014) assign each word a human-rated valence and combine them with rules. **Statistical classifiers** learn word weights from labelled data using bag-of-words or TF-IDF vectors: Naive Bayes is a generative model that applies Bayes' rule under a word-independence assumption, while Logistic Regression and Support Vector Machines are discriminative linear models. **Embedding-based** methods represent a sentence as a dense vector — an average of static word vectors, or the contextual token representations produced by a pre-trained Transformer (RoBERTa, Liu et al., 2019) — and train a linear classifier on top. An **ensemble** averages the probabilities of two complementary models.

4.2 Algorithm(s) Used (e.g., Naive Bayes, LSTM, BERT, Transformers)

Table 4.1: Approaches implemented and compared

#	Approach	Features	Classifier / rule
1	VADER	7,500-entry valence lexicon + 5 heuristics	compound ≥ 0.05 → positive
2	Multinomial Naive Bayes	Word counts (bag-of-words)	Bayes' rule, Laplace smoothing α
3	Logistic Regression	TF-IDF (sublinear tf)	Sigmoid of weighted sum, L2 penalty C
4	Linear SVM	TF-IDF (sublinear tf)	Maximum-margin hyperplane, hinge loss, C
5	Static word vectors + LR	Mean of 300-d spaCy vectors	Logistic Regression
6	RoBERTa embeddings + LR	Mean of 768-d RoBERTa last layer (frozen)	Logistic Regression
7	Ensemble	TF-IDF + RoBERTa	Average of the two LR probabilities

4.3 Architecture Diagram / Layer-Wise Design

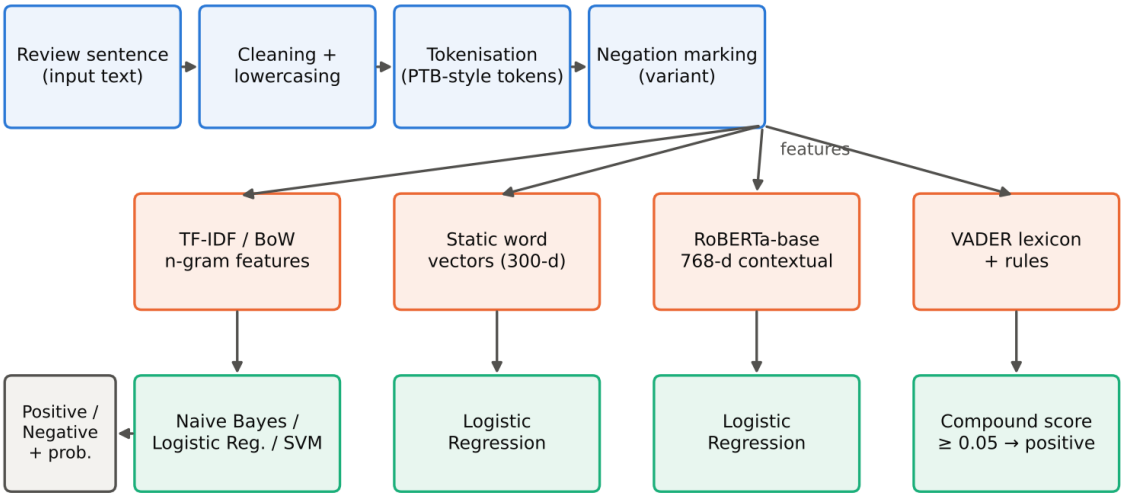


Figure 4.1: Architecture of the sentiment-analysis system: shared preprocessing, four feature extractors and their classifiers

The system has six components: (1) **input** of a review sentence; (2) **preprocessing** (cleaning, lowercasing, tokenisation, optional negation marking); (3) **feature extractors** (bag-of-words/TF-IDF, static vectors, RoBERTa, VADER lexicon); (4) **classifiers** (Naive Bayes, Logistic Regression, SVM, VADER rule); (5) **ensemble** layer; and (6) **output** of label, probability and explanation. RoBERTa-base itself is a 12-layer Transformer encoder: each layer applies 12-head self-attention followed by a feed-forward network with residual connections and layer normalisation, producing a 768-dimensional vector for every sub-word token of the 50,265-token byte-level BPE vocabulary.

4.4 Data Flow in the Model

- **Step 1 - Input:** a sentence such as 'it is not a good film .'
- **Step 2 - Cleaning:** lowercase, remove HTML remnants, collapse whitespace.
- **Step 3 - Tokenisation:** split into tokens (the corpus already separates punctuation and 'n't').
- **Step 4 - Feature extraction:** sparse TF-IDF vector over 19,023 unigrams, and/or a 768-d RoBERTa embedding (mean of sub-word states).
- **Step 5 - Classification:** each Logistic Regression outputs P(positive).
- **Step 6 - Ensemble and decision:** average the probabilities; positive if ≥ 0.5.
- **Step 7 - Explanation:** list the words whose weight × TF-IDF value contributed most.

4.5 Mathematical Formulation (if applicable)

TF-IDF: $w(t, d) = (1 + \log tf(t, d)) \times (\ln((1 + N) / (1 + df(t))) + 1)$, then L2-normalised.

Multinomial Naive Bayes: $\hat{y} = \text{argmax}_c [\log P(c) + \Sigma_t \text{count}(t, d) \cdot \log P(t \mid c)]$, with $P(t \mid c) = (\text{count}(t, c) + \alpha) / (\Sigma_{t'} \text{count}(t', c) + \alpha|V|)$.

Logistic Regression: $P(y = 1 \mid x) = \sigma(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$; trained by minimising cross-entropy + $\|w\|^2 / (2C)$.

Linear SVM: minimise $\frac{1}{2}\|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b))$.

VADER compound score: normalised sum of rule-adjusted valences s, compound = $s / \sqrt{(s^2 + 15)}$.

Self-attention (RoBERTa): $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$; sentence embedding $e = (1/n) \sum_i h_i$ over the n sub-word states h_i of the last layer.

Metrics: Precision = TP / (TP + FP); Recall = TP / (TP + FN); F1 = 2PR / (P + R); Accuracy = (TP + TN) / N; ROC-AUC = area under the true-positive vs false-positive rate curve.

4.6 Why This Model Suits the Selected NLP Task

Short, opinionated sentences contain strong lexical cues ('dull', 'wonderful'), which linear models on TF-IDF features capture efficiently and interpretably. Contextual embeddings encode word order and context, which should help with negation and contrast, and they transfer knowledge from pre-training, which helps when labelled data are scarce. Combining the two in an ensemble uses the strengths of both, while the lexicon baseline shows what is possible without any training data.

5. Dataset and Preprocessing

5.1 Dataset Description (Source, Size, Structure)

Table 5.1: Dataset summary

Property	Value
Name / source	Sentence Polarity Dataset v1.0, Pang and Lee (2005); snippets from Rotten Tomatoes reviews (public, via the Cornell movie-review data)
Size	10,662 sentences: 5,331 positive, 5,331 negative (balanced)
Structure	Two text files (one sentence per line), already lowercased and tokenised
Length	Average 21.0 words, median 20, maximum 59
Vocabulary	21,424 distinct tokens
Split	8,529 training (80%) / 2,133 test (20%), stratified, seed 42

5.2 Data Cleaning Steps

- Read files as UTF-8, ignoring undecodable bytes from the original Latin-1 encoding.
- Removed empty lines and HTML remnants with regular expressions; collapsed repeated whitespace.
- Lowercased all text (already lowercase in the source) so that features are case-insensitive.
- Checked class balance (exactly 50/50), so accuracy is a meaningful headline metric.

5.3 Tokenization, Lemmatization, Stopwords Removal

The corpus is pre-tokenised in Penn-Treebank style (punctuation and clitics such as 'n't' and "s' are separate tokens), so whitespace tokenisation is used for the bag-of-words models; RoBERTa uses its own byte-level BPE sub-word tokeniser. Four preprocessing variants were

compared with 5-fold cross-validation on the training set, each with its own tuned regularisation strength C:

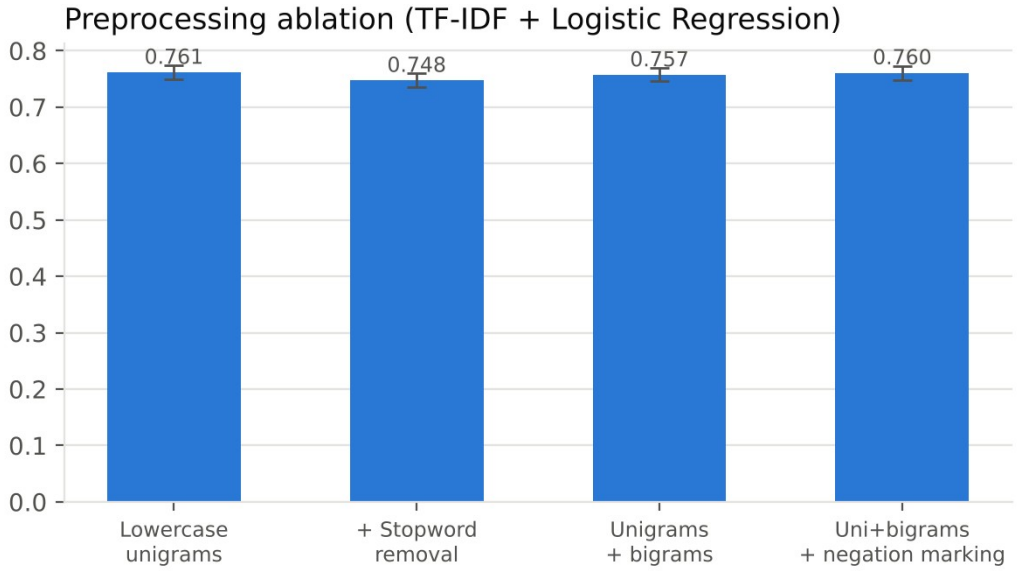


Figure 5.1: Preprocessing ablation (5-fold CV accuracy, mean ± std)

Stopword removal hurts (76.1% → 74.8%) because standard stopword lists contain sentiment-bearing function words such as 'not', 'no', 'too' and 'very'; it was therefore not used. Lemmatisation was not applied for the same reason: inflections like 'bored/boring' carry different sentiment. Negation marking (prefixing NOT_ to every word after a negation until the next punctuation, e.g. 'not a good film' → 'not NOT_a NOT_good NOT_film') reached 76.0%, statistically tied with plain unigrams (76.1%, std ≈ 1.2 points). Following the protocol, the best cross-validation variant (lowercase unigrams) was selected for the classical models.

5.4 Train/Test Split Strategy

A stratified 80/20 split (seed 42) gives 8,529 training and 2,133 test sentences with the same class balance. All hyperparameters (Naive Bayes α; C for Logistic Regression and SVM; C for the embedding classifiers) and the preprocessing variant were chosen by 5-fold stratified cross-validation on the training set only. The test set was used once, for the final evaluation.

5.5 Feature Engineering or Embedding Approach (TF-IDF, Word2Vec, BERT Embeddings)

- **Bag-of-words counts** (Naive Bayes) and **TF-IDF** with sublinear term frequency (LR, SVM): 19,023 unigram features.
- **Static word embeddings:** mean of spaCy en_core_web_md 300-d vectors (trained on OSCAR, Wikipedia, OpenSubtitles and news), standardised.
- **Contextual embeddings:** mean of RoBERTa-base last-layer states (768-d), frozen, standardised; encoding all 10,662 sentences took about 9 minutes on CPU.
- **Lexicon features:** VADER valence scores (no training).

6. Implementation Methodology

6.1 Tools, Libraries, and Environment Used

Table 6.1: Tools and environment

Item	Details
Language / OS	Python 3.13 on Linux; CPU only (2 cores, 8 GB RAM), no GPU
NLP libraries	NLTK (VADER), spaCy 3.8 (en_core_web_md, en_core_web_trf / RoBERTa-base), PyTorch 2
Machine learning	scikit-learn (CountVectorizer, TfidfVectorizer, MultinomialNB, LogisticRegression, LinearSVC, GridSearchCV, StratifiedKFold), SciPy (binomial test)
Visualisation	matplotlib
Project files	data/ (corpus), run_experiments.py, posthoc_negation.py, make_figures.py, predict.py (application), results/ (metrics, cached embeddings), figures/

6.2 Model Implementation Steps

- Loaded and inspected the corpus; created the stratified train/test split.
- Ran the preprocessing ablation with cross-validated Logistic Regression and chose the feature set.
- Evaluated VADER directly on the test set (no training).
- Tuned and trained Naive Bayes, Logistic Regression and Linear SVM with 5-fold grid search.
- Extracted static and RoBERTa sentence embeddings for all sentences and trained tuned Logistic Regression classifiers on them.
- Built the probability-averaging ensemble; computed metrics, confusion matrices, learning curves, McNemar's test and accuracy on linguistic slices.
- Packaged the best models into the predict.py application.

6.3 API Integration (if used)

VADER is called through NLTK's SentimentIntensityAnalyzer.polarity_scores(text), which returns negative, neutral, positive and compound scores. RoBERTa embeddings come from the spaCy pipeline API: nlp.pipe(texts) runs the Transformer and doc._.trf_data.last_hidden_layer_state exposes the last-layer token states, which are mean-pooled. The application in Section 3 wraps both behind a single analyse(text) function; the same function could call a cloud API such as Google Cloud Natural Language for comparison.

6.4 Pseudocode / Workflow Diagram

```
procedure TRAIN():
    X_train, X_test ← stratified_split(corpus, 0.8, seed = 42)
    for each preprocessing variant v: cv_acc[v] ← 5-fold CV of TF-IDF + LR (tune C)
    v* ← argmax cv_acc
    for each model m in {NB, LR, SVM}: m ← GridSearchCV(features(v*), params(m)).fit(X_train)
```

```
    E ← RoBERTa_mean_embeddings(all sentences)        # cached
    rob ← GridSearchCV(StandardScaler + LR).fit(E[train])

procedure PREDICT(sentence):
    p1 ← LR.predict_proba(TFIDF(tokenise(clean(sentence))))
    p2 ← rob.predict_proba(RoBERTa_mean_embedding(sentence))
    p ← (p1 + p2) / 2
    return ('positive' if p ≥ 0.5 else 'negative', p, top_contributing_words)
```

6.5 Hyperparameters and Training Configuration

Table 6.2: Hyperparameters

Model	Search space	Selected (5-fold CV)
Multinomial NB	$\alpha \in \{0.1, 0.3, 0.5, 1.0\}$	alpha = 1.0
Logistic Regression (TF-IDF)	$C \in \{1, 4, 10, 30\}$	C = 4
Linear SVM (TF-IDF)	$C \in \{0.1, 0.3, 1, 3\}$	C = 0.3
Static vectors + LR	$C \in \{0.01 \ldots 1\}$	C = 0.01
RoBERTa + LR	$C \in \{0.01 \ldots 1\}$	C = 0.03
VADER	standard threshold	compound ≥ 0.05
Ensemble	-	equal-weight average of LR and RoBERTa-LR probabilities

No network weights were fine-tuned; RoBERTa was used as a frozen feature extractor.

7. Results and Evaluation

7.1 Performance Metrics (Accuracy, F1-Score, BLEU, Precision/Recall, etc.)

Table 7.1: Results on the held-out test set (n = 2,133); best in bold

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Tuning + train (s)
VADER (lexicon)	0.642	0.628	0.696	0.660	0.690	-
Multinomial Naive Bayes	0.774	0.778	0.766	0.772	0.846	2.1
Logistic Regression (TF-IDF)	0.775	0.766	0.792	0.779	0.858	2.5
Linear SVM (TF-IDF)	0.779	0.772	0.792	0.782	0.857	2.6
Static word vectors + LR	0.696	0.702	0.679	0.691	0.772	8.4
RoBERTa embeddings + LR	0.753	0.767	0.727	0.746	0.835	8.3
Ensemble (TF-IDF LR + RoBERTa LR)	**0.808**	**0.814**	**0.799**	**0.806**	**0.883**	-

Training time includes the 5-fold grid search. Embedding extraction time is not included (RoBERTa: about 9 minutes for the whole corpus on CPU).

7.2 Visualization of Results (Confusion Matrix, Graphs)

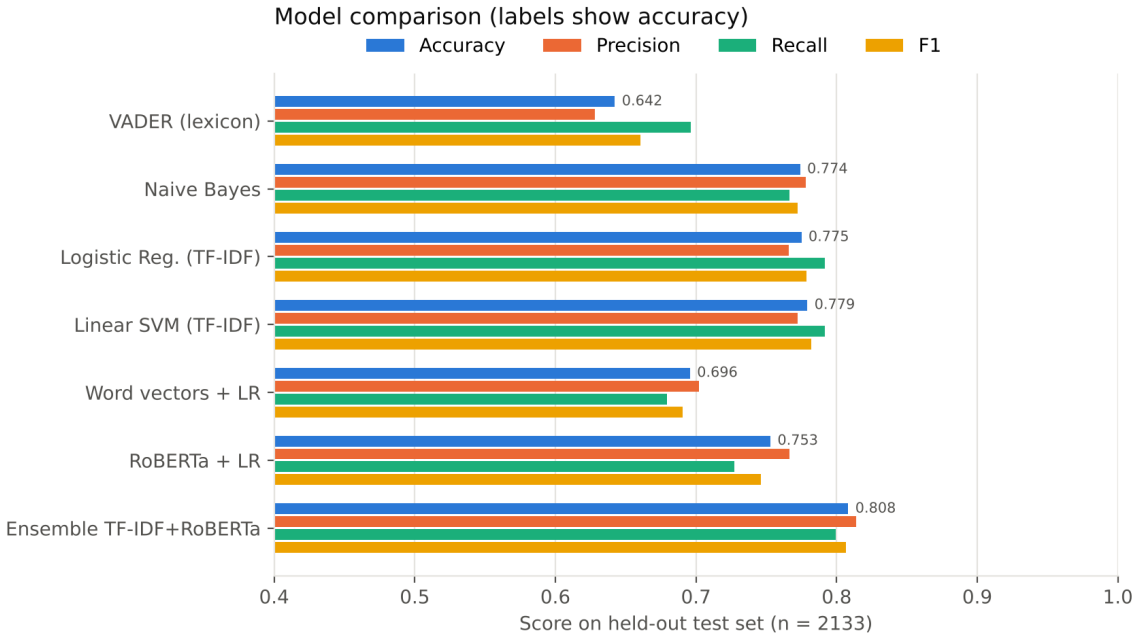


Figure 7.1: Accuracy, precision, recall and F1 of all models

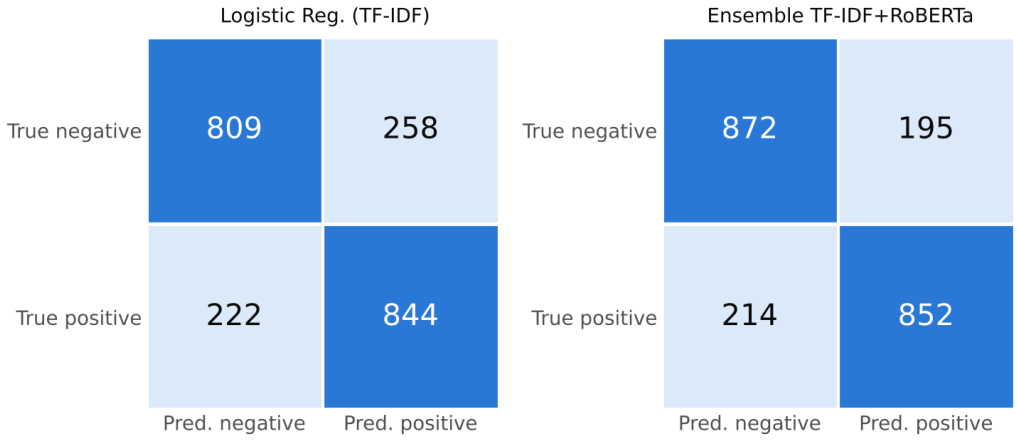


Figure 7.2: Confusion matrices of Logistic Regression (TF-IDF) and the ensemble

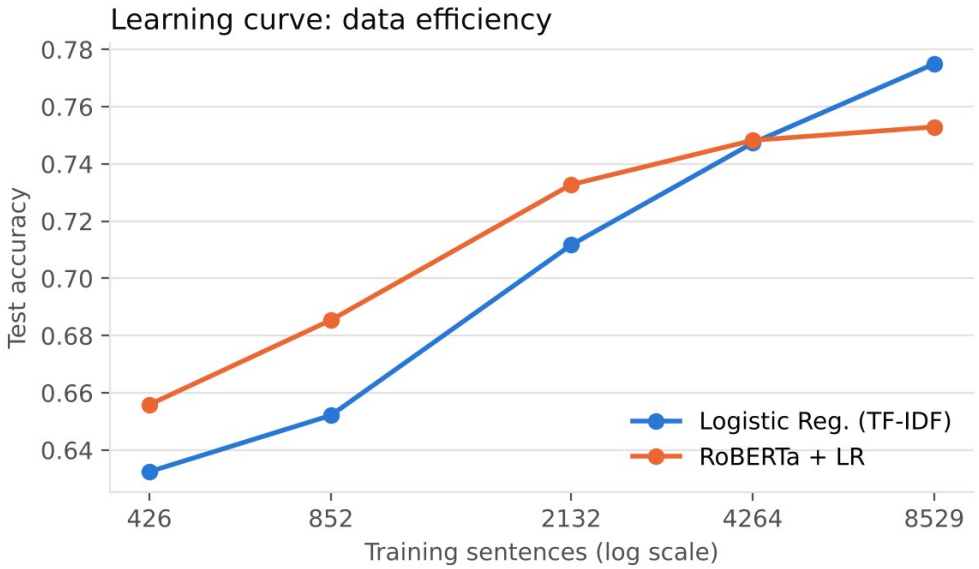


Figure 7.3: Test accuracy as a function of training-set size

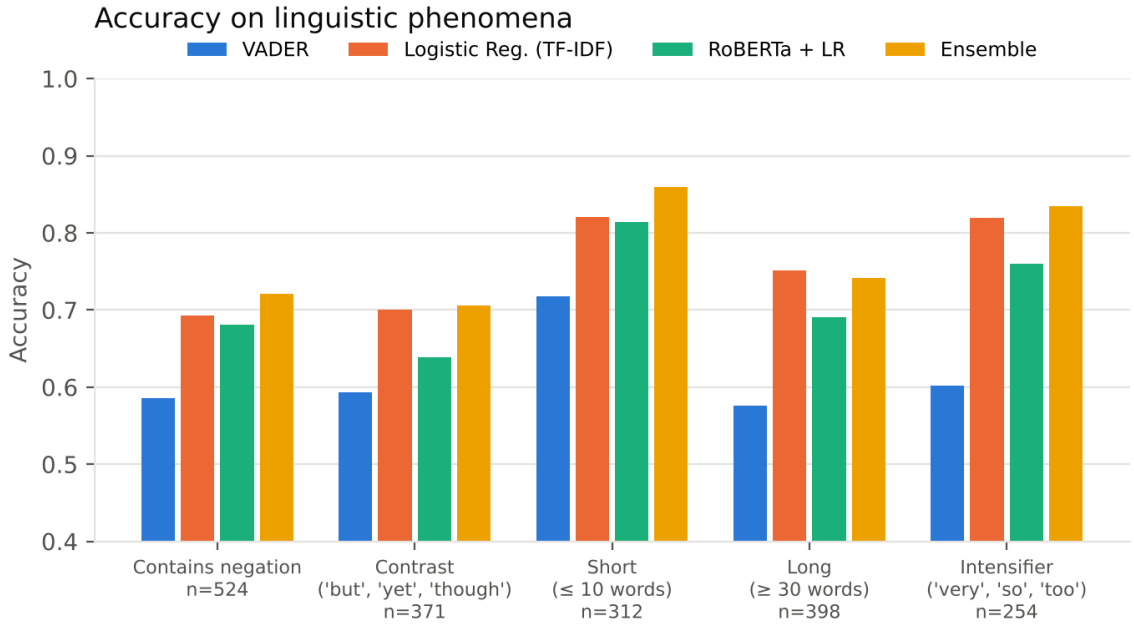


Figure 7.4: Accuracy on sentences with specific linguistic patterns

7.3 Qualitative and Quantitative Analysis

Quantitative observations:

- **Observation 1:** The ensemble of TF-IDF Logistic Regression and RoBERTa Logistic Regression is the best model: accuracy 0.808, F1 0.806, ROC-AUC 0.883, about 2.9 points above the best single model. The two models make different mistakes, so averaging helps.
- **Observation 2:** The classical linear models are strong and nearly identical: Linear SVM 0.779, Logistic Regression 0.775, Naive Bayes 0.774. McNemar's test between SVM and Logistic Regression (26 vs 17 discordant sentences, p = 0.22) shows the difference is not statistically significant.
- **Observation 3:** Frozen RoBERTa embeddings alone (0.753) are below TF-IDF with the full training set, but the learning curve shows they are better with little data: with 426

training sentences RoBERTa reaches 0.656 vs 0.632 for TF-IDF. The curves cross at about 4,000 sentences, after which TF-IDF keeps improving while the frozen embedding saturates.

- **Observation 4:** Averaged static word vectors (0.696) lose word identity and order; averaging dilutes the few sentiment words in a sentence.
- **Observation 5:** VADER, with no training, reaches only 0.642. 17.9% of test sentences get a near-zero (neutral) score because film-review vocabulary ('engrossing', 'tedious', 'pretentious') is missing from or weak in its general-purpose lexicon.
- **Observation 6:** Linguistic patterns matter: on sentences with negation, accuracy drops to 0.693 for TF-IDF LR and 0.586 for VADER; on contrast sentences ('but', 'yet') RoBERTa drops to 0.639. Short sentences are easiest (ensemble 0.859).
- **Observation 7 (post-hoc, not used for model selection):** the negation-marked uni+bigram Logistic Regression, tied in cross-validation, scores 0.795 on the test set and 0.742 on negation sentences (vs 0.693), and it correctly flips 'it is not a good film' to negative. Explicit negation handling is the most promising cheap improvement.

Qualitative analysis - demonstrated test cases:

Table 7.2: Demonstrated test cases (P(positive) for classifiers, compound score for VADER; ✓ correct, ✗ wrong)

#	Sentence	Gold	TF-IDF LR	RoBERTa LR	LR + negation	VADER
TC1	a masterpiece of quiet , heartbreaking performances .	Pos	✓ 0.99	✓ 1.00	✓ 1.00	✓ +0.27
TC2	the plot is predictable and the dialogue is painfully flat .	Neg	✓ 0.04	✓ 0.17	✓ 0.01	✓ -0.53
TC3	not bad at all -- i actually enjoyed it .	Pos	✗ 0.14	✗ 0.20	✗ 0.11	✓ +0.73
TC4	it is not a good film .	Neg	✗ 0.89	✓ 0.38	✓ 0.45	✓ -0.34
TC5	the visuals are stunning , but the story never comes alive .	Neg	✗ 0.54	✓ 0.37	✓ 0.08	✓ -0.24
TC6	i wanted to like this movie , but i could n't .	Neg	✓ 0.09	✓ 0.19	✓ 0.08	✗ +0.19
TC7	oh great , another two hours of my life i will never get back .	Neg	✓ 0.45	✓ 0.36	✓ 0.37	✗ +0.62
TC8	the cast is charming and the script is sharp and funny .	Pos	✓ 0.97	✓ 0.67	✓ 0.99	✓ +0.77

7.4 Error Analysis

The most confident mistakes of the deployed TF-IDF Logistic Regression on the test set fall into four groups:

- **Contrast and concession:** "the result is more depressing than liberating , but it's never boring ." is labelled positive by the annotators but the model gives P(positive) = 0.02, because 'depressing' and 'boring' dominate while 'never' and 'but' are ignored by a bag-of-words model.

- **Double negation / hedged praise:** "halloween : resurrection isn't exactly quality cinema , but it isn't nearly as terrible as it cold have been ." (gold positive, P = 0.03).
- **Faint praise read as positive:** "a solid film . . . but more conscientious than it is truly stirring ." (gold negative, P = 0.92) — 'solid' is a strong positive feature, but the second clause makes the review negative.
- **Sarcasm and out-of-domain phrasing:** TC7 (sarcasm) is caught only by chance by the classifiers, and a casual sentence such as 'The acting was superb and the story kept me hooked' was misclassified by the demo application because function words ('was', 'me', 'kept') carry small negative weights learned from critic snippets.

7.5 Comparison Between Approaches (If multiple methods used)

Table 7.3: Comparison of approaches

Approach	Strengths	Weaknesses	Verdict
VADER	No training; fast; handles some negation and intensifiers	Generic lexicon; many neutral scores; lowest accuracy	Baseline only
Naive Bayes	Fastest to train; robust with small data	Independence assumption	Good baseline
LR / Linear SVM (TF-IDF)	Accurate, interpretable weights, ~0.02 ms per sentence	Blind to word order; negation and contrast errors	Best single model; deploy in fast mode
Static vectors + LR	Dense, small	Averaging blurs sentiment words	Not recommended
RoBERTa + LR	Context-aware; best with little data	Slow on CPU; frozen features saturate	Useful in ensemble
Ensemble	Highest accuracy, F1 and AUC	Needs Transformer at inference	**Best overall**

8. Discussion

8.1 Interpretation of Results

On a balanced sentence-level benchmark, simple linear models on TF-IDF features remain hard to beat (about 77.9% accuracy), which is consistent with published results on this dataset for linear models. Using a large pre-trained Transformer as a frozen feature extractor does not automatically help; its value shows when labelled data are scarce and when its predictions are combined with lexical models (80.8%). Fine-tuning the Transformer end-to-end would be expected to perform best, but requires a GPU.

8.2 Strengths and Weaknesses of the Model

Table 8.1: Strengths and weaknesses

Strengths	Weaknesses
Best model improves accuracy and AUC over every single model	Errors on contrast, double negation and sarcasm remain
Fast mode is interpretable and runs in microseconds	Binary labels cannot express mixed opinions

Rigorous protocol: CV tuning, single test use, significance test	Domain-specific: trained only on critic snippets
Works on CPU without fine-tuning	RoBERTa features are frozen, not adapted to the task

8.3 Challenges Encountered During Implementation

- No GPU: RoBERTa had to be used as a frozen encoder, and embedding 10,662 sentences took about 9 minutes, so embeddings were cached to disk.
- A first preprocessing comparison used one fixed C for all variants, which unfairly penalised the larger bigram feature sets; it was corrected by tuning C per variant.
- Tokenisation of user input had to be made consistent with the corpus (splitting 'n't' and punctuation) for the application to work.
- VADER's default threshold leaves many sentences neutral; the binary mapping (neutral → negative) lowers its recall of negatives.

8.4 Impact of Dataset Quality and Preprocessing

Because the dataset is balanced and clean, accuracy is a fair metric and differences between models are meaningful. Preprocessing choices had measurable effects: removing stopwords cost about 1.3 points of cross-validated accuracy, and negation marking improved accuracy on negated sentences by about 5 points. The narrow domain (professional film critics) limits generalisation to casual reviews.

8.5 How API or Application Enhances the NLP Task

Wrapping the models in an application adds explanations (top contributing words), a choice between fast and accurate modes, and a side-by-side lexicon baseline, which makes the system useful to non-experts and transparent about why it decided. Off-the-shelf APIs such as VADER give instant results without training data, while the trained models show the gain from domain-specific labelled data.

9. Conclusion

9.1 Summary of the Work Done

Seven sentiment-analysis approaches were implemented and compared on 10,662 movie-review sentences. The ensemble of TF-IDF Logistic Regression and RoBERTa embeddings achieved the best accuracy (80.8%), F1 (0.806) and ROC-AUC (0.883); Linear SVM was the best single model (77.9%); VADER without training reached 64.2%. Preprocessing ablations, learning curves, significance testing and linguistic error analysis were carried out, and a working command-line analyser was delivered.

9.2 Key Learnings and Insights

- Stopword removal can hurt sentiment analysis: negations and intensifiers are function words.

- Pre-trained Transformer embeddings shine with little labelled data; with enough data, simple features catch up.
- Combining complementary models is a cheap, reliable gain.
- Evaluation should go beyond accuracy: slices, significance tests and examples reveal what models actually understand.

9.3 Real-World Applicability

The fast mode is suitable for high-volume monitoring of reviews or social-media posts, and the accurate mode for lower-volume, higher-stakes analysis such as customer-escalation triage. The same pipeline can be retrained on any labelled review data (products, restaurants, apps).

9.4 Future Improvements or Extensions

- Fine-tune RoBERTa or DistilBERT end-to-end on a GPU, or use parameter-efficient fine-tuning (LoRA, Unit 3).
- Add explicit negation scope detection and contrast-aware weighting of the clause after 'but'.
- Compare zero-shot and few-shot prompting of an instruction-tuned LLM.
- Extend to three classes (with neutral) and aspect-based sentiment (acting, story, music).
- Support Indian languages and code-mixed (Hinglish / Kanglish) reviews.

10. References

[1] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft, 2024 (Ch. 4: Naive Bayes and Sentiment Classification). (Course textbook)

[2] B. Pang and L. Lee, "Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales," Proc. ACL, 2005. (Sentence polarity dataset v1.0)

[3] B. Pang, L. Lee and S. Vaithyanathan, "Thumbs up? Sentiment Classification using Machine Learning Techniques," Proc. EMNLP, 2002.

[4] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text," Proc. ICWSM, 2014.

[5] S. Das and M. Chen, "Yahoo! for Amazon: Extracting Market Sentiment from Stock Message Boards," Proc. APFA, 2001. (Negation marking)

[6] S. Wang and C. D. Manning, "Baselines and Bigrams: Simple, Good Sentiment and Topic Classification," Proc. ACL, 2012.

[7] Y. Kim, "Convolutional Neural Networks for Sentence Classification," Proc. EMNLP, 2014.

[8] R. Socher et al., "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank," Proc. EMNLP, 2013.

[9] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.

[10] A. Vaswani et al., "Attention Is All You Need," Advances in NeurIPS, 2017.

[11] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," Psychometrika, vol. 12, pp. 153-157, 1947.

[12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, 2011; S. Bird, E. Klein and E. Loper, Natural Language Processing with Python (NLTK), O'Reilly, 2009.

[13] L. Tunstall, L. von Werra and T. Wolf, Natural Language Processing with Transformers, O'Reilly, 2022. (Course reference book)

[14] E. M. Bender et al., "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" Proc. ACM FAccT, 2021.